Vani × Things Alive IoT Platform — Integration Guide for the Platform Team

Version: Nov 3, 2026 · **Status:** Vani side LIVE and proven end-to-end (first live test call Nov 3, 2026, 10:18 PM CT) **Audience:** Things Alive IoT Platform engineering team **Companion spec:** `ThingsAlive_Vani_MachineLane_Spec_20261103.md` (§3 is the contract this guide operationalizes; if anything here and the code disagree, the live API behavior in this guide wins — it was verified against the running service)

---

1. What this is

Vani is the Things Alive voice agent (Twilio + Deepgram, hosted on Railway). The **machine lane** lets the IoT Platform make Vani call a customer about a specific machine:

- **Outbound (platform → Vani → customer):** when the platform raises an alert (or fires a trigger action), it POSTs one API call to Vani with the machine context. Vani's rules engine decides — dial now, defer, or suppress — and owns the call, the retries, the voicemail, and the conversation. The customer sees caller ID **+1 817-809-3797** ("Things Alive AI").
- **Outcome (Vani → platform):** after each call attempt, Vani POSTs an outcome callback to a URL the platform provides.
- **Inbound (customer → Vani):** a registered customer who calls the 817 number is recognized by their number; Vani offers to review their machines using the latest **account snapshot** the platform has pushed.

The platform team's work is four steps: **(1) add the client, (2) add equipment from the library, (3) add test data, (4) wire the API.** A ready-made test client (Step 1) already exists on the Vani side — use it for the first end-to-end run.

Base URL (all endpoints): `https://vani-service-production.up.railway.app`

---

2. Credentials — read first

- The platform authenticates with a dedicated **platform token**: header `Authorization: Bearer <PLATFORM_TOKEN>` on every `/platform/*` call.
- The token is provided by Deepak out-of-band. It is **not** in this document, must live in the platform's server-side secret store, must never appear in client-side code, repos, tickets, or logs, and is different from Vani's admin API token (the platform team does not need and will not receive the admin token).

- Missing or wrong token → `401`. There is no other auth mechanism and no IP allowlist today.

---

3. Step 1 — Add the USA test client

3.1 On the Vani side — ALREADY DONE (Nov 3, 2026)

This exact account is registered in Vani's customer registry and was used for the first live test call:

Field	Value
`account_id`	`usa-test-01`
`account_name`	`USA Test Client`
`contact_name`	`Deepak Panguluri`
`registered_numbers`	`+12012031340` (primary)
`machines`	`dg-01` — "Diesel Generator DG-01", type: Diesel generator, site: Grapevine, TX
`timezone`	`America/Chicago`
`status`	`active`

Registrations are performed by Vani operations (Deepak) from the Command Center's Admin tab — the platform team never writes to the registry directly. For a new client, send Deepak the data in the shape of §3.3 and he registers it.

3.2 On the platform side — the team's task

Create the same client in the IoT Platform so platform records and Vani records join up:

1. Create a customer/tenant: **USA Test Client**.
2. Contact: **Deepak Panguluri**, phone **+1 201-203-1340** (store E.164: `+12012031340`), timezone **America/Chicago**.
3. The platform's record for this client must carry the Vani `account_id` **`usa-test-01`** as an external reference — every API call uses it as the join key.
4. Set the client's **callback URL** (see §6.3) and tell Deepak, so it can be stored on the Vani-side registration (per-trigger override is also supported).

3.3 Registration data shape (for any future client)

```
{
  "account_id": "usa-test-01",
  "account_name": "USA Test Client",
  "contact_name": "Deepak Panguluri",
  "registered_numbers": [{ "number": "+12012031340", "primary": true }],
  "machines": [
    { "machine_id": "dg-01", "name": "Diesel Generator DG-01",
      "type": "Diesel generator", "site": "Grapevine, TX" }
  ],
  "timezone": "America/Chicago",
  "callback_url": "https://<platform-host>/vani/outcome-callback",
  "status": "active"
}
```

Registry rules that will reject a registration (Vani answers `422` with the reason):

- `account_id`, `account_name`, `contact_name` are required.
- `registered_numbers` must be a non-empty list; numbers are normalized to E.164; **a number already registered to another account is a collision, never a silent overwrite.**
- Each machine needs a `machine_id` and a `name`. The `name` is **spoken aloud** by Vani — write it the way a human would say it ("Diesel Generator DG-01", not `DG_01_GENSET`).
- `timezone` must be a valid IANA name (quiet hours are evaluated in the customer's timezone).
- `status` is `active` or `alert_calls_off` (the per-account opt-out).

---

4. Step 2 — Add equipment from the library

On the platform side:

1. Onboard the test equipment **from the machine-knowledge library**: choose the library template for a **diesel generator** (the library supplies the equipment model, signal bindings, and the prediction/diagnostic definitions — do not hand-build a bare equipment record; the library template is what gives the alert in Step 3 its meaning).
2. Name it **Diesel Generator DG-01**, site **Grapevine, TX**.
3. Bind it to the Vani machine identity: `machine_id` **`dg-01`**. The mapping rule for all future equipment: **the platform must send, in every trigger, the `machine_id` and spoken `name` exactly as registered on the Vani side (§3.3).** If the platform's internal equipment ID differs, keep a mapping — Vani does not resolve platform-internal IDs.
4. After creating the client and its equipment, push the account snapshot (§6.4) so inbound calls can review this machine.

---

5. Step 3 — Add test data

Goal: a realistic alert for Vani to call about — the same one already proven live on Nov 3:

1. Feed the equipment test/simulated telemetry (device simulator or telemetry injection — whichever the platform uses for demo data) so the library's prediction has something real to work from:
   - **Main bearing temperature:** trending upward — roughly **+9 °C over 72 hours**.
   - **Vibration:** slightly elevated.
   - **Load:** running at ~62%.
   - Health context: **health score 71/100**, last service **42 days ago**.
2. Configure/confirm the alert rule so this condition raises an alert whose severity maps to **critical** (mapping table in §6.1): the prediction is *bearing failure risk rising, predicted window 10–14 days if the trend continues*, recommended action *schedule an inspection within the week; no immediate shutdown needed*.
3. When the alert fires, the platform builds the **machine packet** (the `context` object, §6.2) from the library/health-card data — the packet is the *only* thing Vani may speak from. Vani never invents machine facts; whatever the platform wants the customer to hear must be in the packet.

---

6. Step 4 — The Vani API integration

6.1 Severity mapping (platform → Vani)

Vani's vocabulary is exactly three severities. Map the platform's native scale once, in the integration layer:

Vani severity	Meaning	Typical platform alerts
`critical`	Call immediately, **including quiet hours**	Predicted failure with a near window, safety-relevant faults, machine down
`warning`	Call in business hours only (deferred outside them)	Degrading health trend, service due, non-urgent threshold breach
`info`	**Never calls** — recorded only	Recoveries, notices, FYI state changes

6.2 Trigger a call — `POST /platform/alert-call`

Headers: `Authorization: Bearer <PLATFORM_TOKEN>`, `Content-Type: application/json`.

Proven payload — this exact call placed the first live test call (Nov 3, 2026):

```
{
  "event_id": "test-20261103-2217",
  "account_id": "usa-test-01",
  "customer_phone": "+12012031340",
  "machine": { "machine_id": "dg-01", "name": "Diesel Generator DG-01" },
  "trigger": {
    "kind": "alert",
    "severity": "critical",
    "title": "Test alert — DG-01 main bearing temperature rising"
  },
  "context": {
    "alert_title": "Test alert — DG-01 main bearing temperature rising",
    "summary": "The prediction library flags the main bearing temperature on Diesel Generator DG-01
trending upward.",
    "machine_status": "Running at 62 percent load",
    "prediction": "Main bearing failure risk rising — predicted window 10 to 14 days if the trend
continues",
    "key_signals": "Bearing temperature up 9 degrees over the last 72 hours; vibration slightly
elevated",
    "recommended_action": "Schedule an inspection within the week; no immediate shutdown needed",
    "health_card": "Health score 71 of 100; last service 42 days ago"
  }
}
```

Response: `202 {"trigger_id": "eb6a26492ada", "event_id": "test-20261103-2217", "call_id": "f89d1ee47b1c", "status": "queued", "disposition": "dialed"}` — the customer's phone rang seconds later.

Field rules:

Field	Rule
`event_id`	**Required. The idempotency key** — one event = one call flow, however many times it is POSTed. Use the platform's alert/event ID.
`account_id`	Required. Must match a registered account exactly (`422` if unknown — register first, §3).
`customer_phone`	Optional cross-check. If sent, it must be one of the account's registered numbers, else `422`. If omitted, Vani calls the account's primary number. **Vani never silently re-routes to an unregistered number.**
`machine`	Required: `{machine_id, name}` as registered (§4).
`trigger.kind`	`alert` or `trigger_action`.

Field	Rule
`trigger.severity`	`critical` | `warning` | `info` (after your mapping, §6.1) — anything else is a `422`.
`trigger.title`	Optional; falls back to the packet's `alert_title`.
`trigger.fired_at`	Optional ISO timestamp of the alert.
`context`	**Required object — the machine packet.** Compose it from library/health-card data as in the example (`alert_title`, `summary`, `machine_status`, `prediction`, `key_signals`, `recommended_action`, `health_card`). Keep values speakable: plain sentences, no codes the customer wouldn't understand.
`callback_url`	Optional per-trigger override of the account's stored callback URL (§6.3).

Responses:

Case	Status	Body (shape)
Dialing now	`202`	`{trigger_id, event_id, call_id, status: "queued", disposition: "dialed"}`
Deferred by rules (e.g. warning in quiet hours)	`202`	`{…, call_id: null, status: "deferred", disposition: "deferred", reason, retry_at}` — Vani dials at `retry_at` on its own; the platform does nothing more.
Suppressed by rules (e.g. `info`, or storm cap reached)	`202`	`{…, call_id: null, status: <policy outcome>, disposition, reason}` — and the outcome callback still fires (§6.3), so the platform's alert history is never left silent.
Duplicate `event_id`	`200`	`{status: "duplicate", event_id, call_id, trigger_id}` — the original reference; **never dials twice**. Safe to retry POSTs.
Bad/missing token	`401`	—
Validation error	`422`	`{"error": "<specific reason>"}` — unknown account, unregistered phone, bad severity, missing packet, etc. Fix the payload; do not retry unchanged.

What the rules engine does with a trigger (Vani-side policy, owned by Deepak, adjustable in the Command Center Admin tab — the platform team does not configure this): severity floor (info

never calls); quiet hours **8 PM–8 AM customer-local** for warning-and-below (critical overrides); retries up to **2 attempts, 45 minutes apart, voicemail on the final attempt**; storm caps **3 calls per machine per day, 6 per account per day**; an account with `alert_calls_off` gets no calls. Deferred triggers are re-evaluated and dialed by Vani automatically.

The conversation: Vani opens by identifying herself as the Things Alive AI voice, states the alert from the packet, and discusses *only* the packet's facts (prediction, signals, recommended action). If the customer asks for a human, she bridges to Deepak. If the call goes to a mailbox, the alert message is delivered by Twilio's text-to-speech channel (deterministic — the exact packet message, not a generated one). Alert calls keep their own ledger and never interfere with any other calling program.

curl version of the proven call:

```
curl -X POST https://vani-service-production.up.railway.app/platform/alert-call \
  -H "Authorization: Bearer $PLATFORM_TOKEN" \
  -H "Content-Type: application/json" \
  -d @alert-payload.json
```

6.3 Receive the outcome — the platform's callback endpoint

The platform exposes one public HTTPS endpoint and registers its URL as the account's `callback_url` (or passes it per trigger). After **each attempt** (including deferred/suppressed dispositions), Vani POSTs:

```
{
  "event_id": "test-20261103-2217",
  "call_id": "f89d1ee47b1c",
  "account_id": "usa-test-01",
  "machine_id": "dg-01",
  "outcome": "completed_spoke_with_customer",
  "summary": "Customer answered and spoke with Vani about Diesel Generator DG-01; no follow-up
booked.",
  "duration_s": 0.0,
  "attempt": 1,
  "called_at": "2026-11-04T04:18:20+00:00"
}
```

- `outcome` enum — exactly one of: `completed_spoke_with_customer` · `voicemail_dropped` · `voicemail_not_delivered` · `no_answer` · `failed`.
- `summary` is one deterministic line written for the platform's alert history — display it as-is.
- **Dedupe on `event_id`** (with `attempt`): Vani retries a failed callback exactly once; a retry can also follow a response the platform actually processed. Return any `2xx` to acknowledge.

- A dead callback endpoint never blocks the call pipeline — but the outcome is then only visible in Vani's records, so keep the endpoint healthy.

6.4 Push the account snapshot — `POST /platform/account-snapshot`

So an inbound customer call can review "my machines," the platform pushes a snapshot whenever the account's machine picture materially changes (equipment added/removed, health status change) and once right after Step 2:

```
{
  "account_id": "usa-test-01",
  "generated_at": "2026-11-03T22:30:00-06:00",
  "machines": [
    { "machine_id": "dg-01", "name": "Diesel Generator DG-01",
      "status": "Running — bearing temperature alert active",
      "health_score": 71 }
  ]
}
```

Response: `200 {"ok": true, "account_id": "usa-test-01", "received_at": "…"}`. Same platform token. Unknown `account_id` → `422`. Vani speaks from the newest snapshot and states its age honestly ("as of this morning's update…") — push fresh snapshots rather than editing history.

6.5 What the platform must NOT do

- Never put the platform token in browser/mobile code, URLs, or logs.
- Never POST a trigger for an unregistered account or an unregistered number — registration is the identity check (§3); the API will `422`, by design.
- Never retry a `422` unchanged, and never "retry" a trigger with a *new* `event_id` for the same alert to force a second call — retries are Vani's job, governed by the rules engine.

---

7. End-to-end acceptance test (run in order)

Using the USA test client — the registered phone is Deepak's, so coordinate the run with him:

1. ☐ Platform client `usa-test-01` mirror exists; callback URL set and shared with Deepak.
2. ☐ Equipment **Diesel Generator DG-01** added **from the library**, bound to `machine_id` `dg-01`; snapshot pushed (§6.4) → `200 ok`.
3. ☐ Test data loaded (§5); the platform raises the bearing-temperature alert.
4. ☐ Platform POSTs the trigger (§6.2) → expect `202`, `disposition: "dialed"`. **The phone rings within seconds**; Vani states the alert using only packet facts (prediction window, +9°

signal, inspection recommendation).

5. ☐ Call ends → the platform's callback endpoint receives the outcome (§6.3) with `outcome: "completed_spoke_with_customer"` after a conversation (or `voicemail_dropped` if it goes to the mailbox) — dedupe on `event_id` works.
6. ☐ Re-POST the same `event_id` → `200 "duplicate"`, **no second ring**.
7. ☐ POST a `warning` trigger after 8 PM customer time → `202 "deferred"` with a `retry_at` at the next window; POST an `info` trigger → suppressed, callback still received, **no ring, ever**.
8. ☐ POST with an unregistered `customer_phone` → `422`; POST without the token → `401`.
9. ☐ Inbound: from the registered number, call **+1 817-809-3797** → Vani recognizes the account and offers to review DG-01 from the snapshot.

Steps 4–6 were already proven from the Vani side on Nov 3 (trigger → dialed → live conversation on the registered number); this run proves the platform's half of the same loop.

---

8. Operations & ownership

- **Calling policy** (severity floor, quiet hours, retries, caps, per-account opt-out) lives on the Vani side and is managed by Deepak in the **SDR Command Center → Admin tab**, which also shows live Twilio/Deepgram/API status and the day's triggers. Platform engineers do not need access for this integration.
- **If a trigger misbehaves:** capture the `event_id`, the full response body, and the timestamp — that trio is enough for Vani operations to trace it in the alert ledger.
- **Changes to this contract** (new fields, new severities, callback changes) go through Deepak — the endpoint is versioned by behavior, and unannounced platform-side changes to payload shape will surface as `422`s, not silent acceptance.

Prepared by Dippy (Deepak's agent) from the live Vani service implementation, Nov 3, 2026. The payload in §6.2 is the verbatim call that placed the first live machine-lane test call.